

校车监控管理系统 (Flutter Windows 桌面端)

目录

1. 项目概述
2. 总体架构设计
3. 模块详细设计
4. UML 建模设计 (类图、时序图、包图)
5. 编译构建设计 (CMake)
6. 核心业务流程
7. 接口与消息机制
8. 异常处理与容错设计
9. 部署与运行说明
10. 总结与扩展规划

1 项目概述

1.1 项目背景

本项目为校车监控管理系统桌面客户端，面向学校管理人员、运维人员使用，实现校车状态查看、人员考勤、路线监控、设备状态管理等业务功能。采用 Flutter 跨平台 UI + Windows 原生 C++ 嵌入 架构：Flutter 负责前端界面与业务交互，原生 Win32 C++ 作为应用宿主，完成窗口管理、系统交互、插件加载、底层消息转发。

1.2 项目目标

1. 实现 Windows 桌面端标准窗口程序，支持高 DPI、系统明暗主题适配；
2. 完成 Flutter 引擎在 Win32 窗口中的嵌入、启动、销毁全生命周期管理；
3. 兼容 Flutter 插件体系，支持原生能力扩展；
4. 提供稳定的消息转发机制，实现系统窗口消息与 Flutter 引擎互通；
5. 保证程序资源合理释放，无内存泄漏、崩溃问题。

1.3 功能范围

1.3.1 基础功能 (原生层)

- Windows 标准窗口创建、显示、缩放、关闭、主题适配；
- Flutter 引擎初始化、视图挂载、插件自动注册；
- 命令行参数解析、控制台输出重定向、字符编码转换；
- COM 组件初始化与资源回收；
- 窗口消息路由、系统事件监听 (字体变更、DPI 变化、窗口大小变化)。

1.3.2 业务功能 (Flutter 层)

- 校车实时状态监控；
- 驾驶员 / 学生考勤管理；
- 行驶路线、定位信息展示；
- 设备告警、日志查看、系统设置。

1.4 技术栈

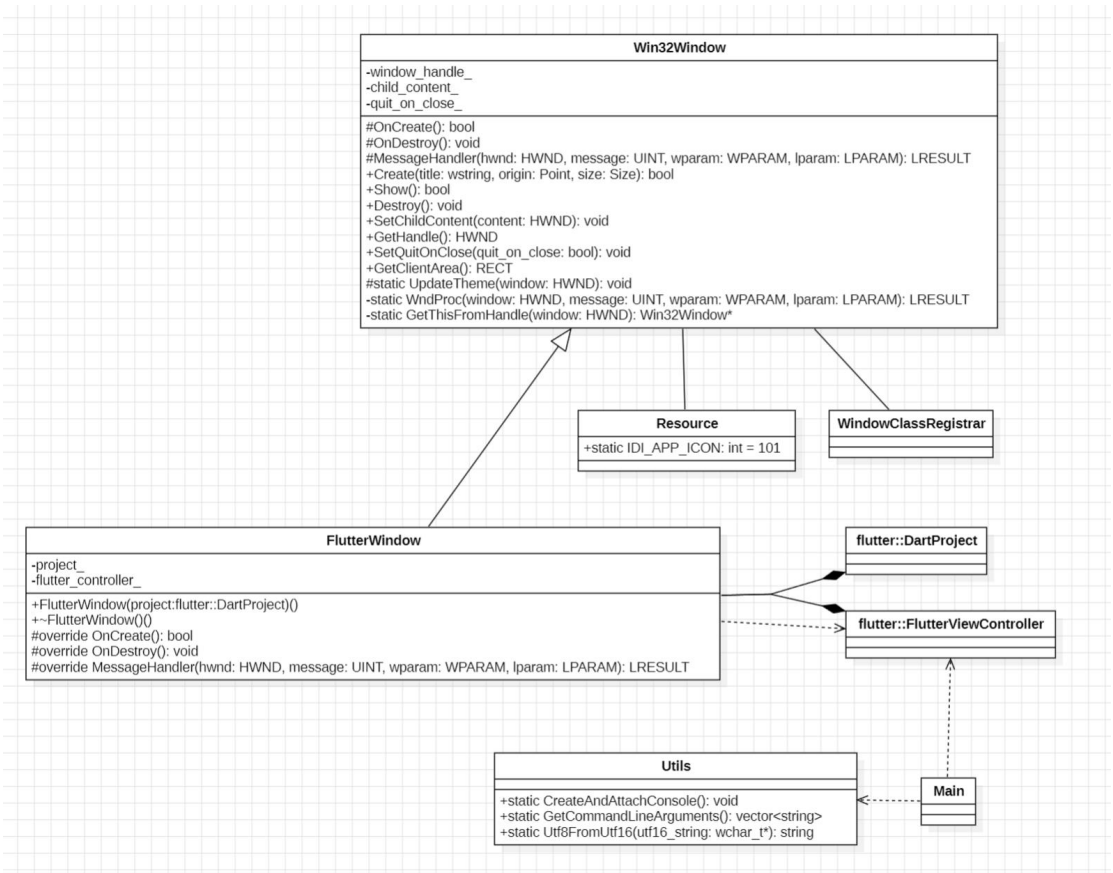
层级	技术 / 框架用途
----	-----------

层级	技术 / 框架用途
应用宿主	Win32 C++ Windows 窗口、系统 API、Flutter 引擎托管
界面层	Flutter 跨平台 UI、业务页面、交互逻辑
构建工具	CMake 跨平台编译配置、依赖管理
建模工具	StarUML UML 类图、时序图、包图设计
编译环境	MSVC Windows C++ 代码编译

2 总体架构设计

2.1 整体架构分层

采用三层架构：应用入口层 → 原生窗口管理层 → Flutter 引擎层，自上而下调用，底层向上回调。



入口层（main.cpp）	程序启动、环境初始化、创建主窗口、消息循环驱动
原生窗口层（Win32Window / FlutterWindow）	基础窗口封装、Flutter 承载窗口、消息分发、生命周期管理

工具支撑层 (utils)	
控制台、编码转换、命令行解析、通用工具函数	
资源&编译层 (resource.h / CMakeLists.txt)	
图标资源、编译配置、依赖链接、Flutter 构建脚本	
Flutter 引擎层 (SDK + 插件)	
Dart 项目、视图控制器、插件注册、渲染引擎、平台通道	

2.2 核心运行流程

1. 系统启动, wWinMain 作为 Windows 程序入口, 初始化控制台、COM 组件;
2. 解析命令行参数, 初始化 DartProject 配置;
3. 创建 FlutterWindow 窗口实例, 继承自基础窗口 Win32Window;
4. 原生窗口创建完成后, 初始化 FlutterViewController 视图控制器;
5. 自动注册所有 Flutter 原生插件, 将 Flutter 视图挂载到原生窗口;
6. 启动 Windows 消息循环, 窗口消息优先转发给 Flutter 引擎处理;
7. 窗口关闭时, 依次释放 Flutter 引擎、原生窗口、COM 资源, 程序退出。

2.3 模块依赖关系

main.cpp: 依赖 FlutterWindow、utils、Flutter SDK;

FlutterWindow: 继承 Win32Window, 组合 DartProject、FlutterViewController;

Win32Window: 依赖资源文件 resource.h、DWM 主题 API、窗口注册器;

utils: 依赖 Windows API、Flutter 输出流接口, 为全局提供通用能力;

CMake: 统一管理所有源码、库依赖、编译规则、Flutter 构建任务。

3 模块详细设计

3.1 入口模块 main.cpp

3.1.1 模块简介

Windows 图形程序标准入口, 函数 wWinMain 为程序启动起点, 负责环境初始化、对象创建、消息循环、资源收尾。

3.1.2 核心代码与逻辑

1. 控制台挂载: 调试 /flutter run 模式下创建控制台, 输出日志;
2. COM 初始化: 为插件、系统组件提供 COM 环境;
3. Dart 项目配置: 指定 Flutter 资源目录 data, 绑定启动参数;
4. 窗口实例化: 创建 FlutterWindow, 设置窗口标题、坐标、尺寸 (1280 × 720);
5. 消息循环: GetMessage/TranslateMessage/DispatchMessage 驱动窗口运行;
6. 资源释放: 程序退出前反初始化 COM。

3.1.3 关键函数说明

wWinMain: Windows 图形应用入口函数;

AttachConsole/CreateAndAttachConsole: 控制台挂载与创建;

CoInitializeEx/CoUninitialize: COM 组件初始化与销毁;

消息循环: Windows 窗口程序核心, 持续监听并分发系统消息。

3.2 基础窗口模块 win32_window.h/win32_window.cpp

3.2.1 模块简介

封装高 DPI 自适应 Win32 基础窗口类, 是所有自定义窗口的父类, 提供窗口创建、销毁、消息处理、主题适配、DPI 缩放等通用能力。

3.2.2 核心类: Win32Window

内部结构体

1. Point: 窗口坐标 (x, y);
2. Size: 窗口尺寸 (宽、高)。

私有属性

window_handle_: 原生窗口句柄;

child_content_: 子窗口句柄 (Flutter 视图);

quit_on_close_: 标记关闭窗口是否退出程序。

核心方法

方法名	作用
Create()	创建原生窗口, 自动适配 DPI 缩放、系统主题
Show()	显示窗口
Destroy()	销毁窗口, 释放系统资源
SetChildContent()	挂载子窗口 (Flutter 视图)
GetClientArea()	获取窗口客户区矩形
MessageHandler	统一窗口消息处理 (大小变化、DPI 变化、激活、销毁)
UpdateTheme	读取系统注册表, 适配 Windows 深色 / 浅色模式

3.2.3 内部辅助类: WindowClassRegistrar

单例类, 负责 Win32 窗口类的注册与注销, 保证全局唯一窗口类, 避免重复注册报错。

3.3 Flutter 承载窗口模块 flutter_window.h/flutter_window.cpp

3.3.1 模块简介

继承自 Win32Window, 专门用于托管 Flutter 视图与引擎, 是原生层与 Flutter 层的桥梁。

3.3.2 核心类: FlutterWindow

私有成员

project_: flutter::DartProject, 保存 Flutter 项目配置;

flutter_controller_: 智能指针管理 FlutterViewController, Flutter

核心控制器。

重写生命周期方法

1. OnCreate (窗口创建)

调用父类完成原生窗口初始化;

根据窗口客户区尺寸创建 Flutter 视图控制器;

校验引擎与视图有效性;

调用 RegisterPlugins 自动注册 Flutter 插件;

挂载 Flutter 视图到原生窗口;

首帧渲染完成后显示窗口, ForceRedraw 防止白屏。

2. OnDestroy (窗口销毁)

主动释放 Flutter 控制器资源;

调用父类销毁原生窗口。

3. MessageHandler (消息处理)

优先将窗口消息交给 Flutter 引擎处理;

监听 WM_FONTCHANGE 系统字体变更, 通知 Flutter 重载字体;

未处理消息转交父类窗口处理。

3.4 工具模块 utils.h/utils.cpp

3.4.1 模块简介

通用工具集, 提供控制台管理、字符编码转换、命令行参数解析, 全局通用, 无状态。

3.4.2 核心函数

1. CreateAndAttachConsole: 分配控制台窗口, 重定向标准输出 / 错误输出;

2. GetCommandLineArguments: 获取 Windows UTF-16 命令行参数, 并转为 UTF-8;

3. Utf8FromUtf16: UTF-16 (Windows 原生编码) 转 UTF-8 (Flutter 引擎编码)。

3.5 资源模块 resource.h

仅定义应用图标资源 ID IDI_APP_ICON = 101, 窗口注册时加载程序图标。

3.6 插件注册模块 generated_plugin_registrant.h

Flutter 自动生成文件, 声明 RegisterPlugins 函数, 用于批量注册所有 Flutter 原生插件, 禁止手动修改。

3.7 编译配置模块 CMakeLists.txt

3.7.1 模块简介

CMake 编译脚本, 管理源码、依赖库、头文件路径、编译规则、Flutter 构建任务。

3.7.2 核心配置项

1. 最低 CMake 版本: 3.14;

2. 编译目标: Runner 可执行程序 (WIN32 图形程序);

3. 源码列表: 包含所有 C++ 源码、资源文件、自动生成插件文件;

4. 依赖库: flutter、flutter_wrapper_app、dwmapi.lib (窗口主题 API);

5. 预处理宏: 关闭 Windows 冲突宏 NOMINMAX, 注入 Flutter 版本信息;

6. 自定义构建任务: flutter_assemble, 触发 Flutter 引擎编译、资源生成。

4 UML 建模设计 (StarUML)

4.1 包图 (Package Diagram)

按工程目录与功能划分模块包, 直观体现工程结构, 可直接在 StarUML 中创建。

4.1.1 包划分

SchoolBus_App (根包)

- └─ Main_Entry 【入口包】 main.cpp
- └─ Win32_Window 【原生窗口包】 win32_window.h/cpp、resource.h
- └─ Flutter_Window 【Flutter 窗口包】 flutter_window.h/cpp
- └─ Utils 【工具包】 utils.h/cpp
- └─ Flutter_SDK 【外部依赖包】 DartProject、FlutterViewController
- └─ Build_Config 【编译配置包】 CMakeLists.txt

4.1.2 包间依赖 (虚线箭头)

1. Main_Entry → 依赖: Flutter_Window、Utils、Flutter_SDK
2. Flutter_Window → 依赖: Win32_Window、Flutter_SDK
3. Win32_Window → 依赖: Win32_Window 内部资源类
4. 所有包 → 依赖: Build_Config

4.2 核心类图 (Class Diagram)

4.2.1 类关系总览

1. 泛化 (继承): FlutterWindow → Win32Window
2. 组合 (强拥有):
 - FlutterWindow 组合 DartProject
 - FlutterWindow 组合 FlutterViewController (unique_ptr 生命周期绑定)
3. 关联: Win32Window 关联内部结构体 Point、Size、WindowClassRegistrar
4. 依赖: main、FlutterWindow、Win32Window 依赖 Utils

4.2.2 类详细定义 (UML 标准格式)

1. Win32Window

plaintext

```
+ Win32Window()
+ ~Win32Window()
+ Create(title:wstring, origin:Point, size:Size): bool
+ Show(): bool
+ Destroy(): void
+ SetChildContent(content:HWND): void
+ GetHandle(): HWND
+ SetQuitOnClose(quit_on_close:bool): void
+ GetClientArea(): RECT
# OnCreate(): bool
# OnDestroy(): void
```

```
# MessageHandler(hwnd:HWND, message:UINT, wparam:WPARAM, lparam:LPARAM):  
LRESULT
```

```
- WndProc(): LRESULT  
- GetThisFromHandle(): Win32Window*  
- UpdateTheme(): void  
- window_handle_: HWND  
- child_content_: HWND  
- quit_on_close_: bool  
+ Point  
+ Size
```

2. FlutterWindow

plaintext

```
+ FlutterWindow(project:DartProject)  
+ ~FlutterWindow()  
# override OnCreate(): bool  
# override OnDestroy(): void  
# override MessageHandler(...): LRESULT  
- project_: DartProject  
- flutter_controller_: unique_ptr<FlutterViewController>
```

3. Utils (静态工具类)

plaintext

```
+ static CreateAndAttachConsole(): void  
+ static GetCommandLineArguments(): vector<string>  
+ static Utf8FromUtf16(str:wchar_t*): string
```

4. WindowClassRegistrar (单例类)

plaintext

```
+ static GetInstance(): WindowClassRegistrar*  
+ GetWindowClass(): wchar_t*  
+ UnregisterWindowClass(): void  
- instance_: WindowClassRegistrar*  
- class_registered_: bool
```

4.3 时序图 (Sequence Diagram)

4.3.1 应用启动时序图

参与者: main、Utils、DartProject、FlutterWindow、Win32Window、FlutterViewController、消息队列

1. main → Utils: CreateAndAttachConsole() 创建控制台
2. main: CoInitializeEx() 初始化 COM
3. main → DartProject: 实例化 DartProject, 加载 data 目录
4. main → Utils: GetCommandLineArguments() 获取参数
5. DartProject: set_dart_entrypoint_arguments() 绑定参数
6. main → FlutterWindow: 构造函数, 传入 DartProject
7. FlutterWindow → Win32Window: Create() 创建原生窗口
8. Win32Window: OnCreate() 基础初始化、主题适配、DPI 适配
9. FlutterWindow → FlutterViewController: 实例化控制器
10. FlutterViewController: 校验 engine /view
11. FlutterViewController: RegisterPlugins() 注册插件
12. FlutterWindow → Win32Window: SetChildContent() 挂载 Flutter 视图
13. FlutterViewController: 设置帧回调 Show() 显示窗口
14. FlutterViewController: ForceRedraw() 强制渲染
15. main → 消息队列: 进入 GetMessage 循环, 持续分发消息

4.3.2 窗口销毁时序图

1. 系统触发 WM_DESTROY 消息
2. FlutterWindow → OnDestroy: 释放 FlutterViewController
3. FlutterWindow → Win32Window: 父类销毁逻辑
4. Win32Window: Destroy() 销毁窗口、注销窗口类
5. main: CoUninitialize() 释放 COM
6. 程序退出

5 编译构建设计

5.1 编译环境要求

操作系统: Windows 10/11 x64

编译工具: Visual Studio 2019+、CMake 3.14+

Flutter 版本: 稳定版 Windows 编译环境就绪

5.2 CMake 核心规则

1. 编译类型: WIN32 图形界面程序, 无控制台黑框 (调试模式可手动开启);
2. 源码组织: 所有 .h/.cpp 均加入编译目标;
3. 外部库依赖:
flutter: Flutter 核心库
flutter_wrapper_app: Flutter C++ 包装层
dwmapi.lib: Windows 桌面窗口管理 (主题、深色模式)
4. 预定义宏:
NOMINMAX: 解决 C++ 标准库与 Windows 宏冲突;
注入 Flutter 版本号相关宏。

5. 构建依赖: flutter_assemble 优先执行, 自动生成 Flutter 引擎、插件代码、资源。

5.3 编译命令

在 Flutter 项目根目录执行标准命令即可自动编译运行:

```
bash
```

```
运行
```

```
# 编译并运行 Windows 客户端
```

```
flutter run -d windows
```

```
# 正式打包发布
```

```
flutter build windows
```

6 核心业务流程

6.1 程序启动全流程

1. 系统启动 → 调用 wWinMain;
2. 初始化调试控制台、COM 组件;
3. 解析命令行参数, 初始化 Flutter 项目配置;
4. 创建 FlutterWindow 窗口, 设置标题、位置、大小;
5. 原生窗口创建、DPI 适配、主题加载;
1. 初始化 Flutter 引擎与视图, 注册所有插件;
2. 挂载 Flutter 视图到原生窗口;
3. 首帧渲染完成后显示窗口, 进入消息循环;
4. Flutter 页面加载完成, 展示校车监控主界面。

6.2 系统事件处理流程

以系统字体变更 (WM_FONTCHANGE) 为例:

1. 系统触发字体变更消息;
2. 消息进入 FlutterWindow::MessageHandler;
3. 优先转交 Flutter 引擎尝试处理;
4. 匹配 WM_FONTCHANGE, 调用 ReloadSystemFonts() 重载字体;
5. 消息处理完成, 返回结果。

6.3 窗口关闭退出流程

1. 用户点击窗口关闭按钮, 触发 WM_DESTROY;
2. Win32Window 捕获消息, 标记退出程序;
3. FlutterWindow::OnDestroy 释放 Flutter 引擎与视图;
4. 销毁原生窗口, 注销窗口类;
5. 退出消息循环, 反初始化 COM;
6. 程序正常退出。

7 接口与消息机制

7.1 窗口消息转发规则

1. 优先级 1: Flutter 引擎 & 插件 优先处理所有窗口消息;
2. 优先级 2: FlutterWindow 自定义消息 (字体变更等);

3. 优先级 3: 父类 Win32Window 处理通用窗口消息 (大小、DPI、激活、销毁);
4. 优先级 4: Windows 默认窗口过程 DefWindowProc。

7.2 原生与 Flutter 通信扩展接口

当前架构支持 Flutter Platform Channel 扩展:

可在 FlutterWindow 中新增 Method Channel / Event Channel 逻辑;

原生 C++ 可主动向 Flutter 推送校车设备状态、告警信息;

Flutter 界面操作可调用原生能力 (串口、硬件、系统权限等)。

8 异常处理与容错设计

8.1 启动异常

1. 原生窗口创建失败: Create 返回 false, 程序直接退出 EXIT_FAILURE;
2. Flutter 引擎 / 视图初始化失败: 终止启动, 避免空指针崩溃;
3. 命令行参数解析失败: 返回空数组, 不影响主流程。

8.2 资源容错

1. 使用 std::unique_ptr 管理 FlutterViewController, 自动析构, 防止内存泄漏;
2. 窗口计数 g_active_window_count: 所有窗口销毁后再注销窗口类, 避免重复注销报错;
3. 字符编码转换增加空指针判断, 防止非法字符串导致崩溃。

8.3 兼容性容错

1. 重定义 DWMWA_USE_IMMERSIVE_DARK_MODE, 兼容低版本 Windows SDK;
2. 动态加载 User32.dll 的 DPI 接口, 接口不存在则跳过, 保证兼容老旧系统。

9 部署与运行说明

9.1 运行环境

- 系统: Windows 10 1903 及以上 / Windows 11
- 依赖组件: Visual C++ 运行库、.NET Framework (系统自带)

9.2 部署方式

1. 执行 flutter build windows, 在 build/windows/x64/release 生成发布包;
2. 将整个发布文件夹拷贝至目标电脑, 完整目录不可拆分 (包含 Flutter 引擎、资源、动态库);
3. 双击 Runner.exe 启动校车监控系统。

9.3 调试方式

1. 命令行执行 flutter run -d windows, 自动拉起调试控制台;
2. 控制台可查看 Flutter 日志、原生输出日志;
3. 可使用 Visual Studio 附加进程调试 C++ 原生代码。

10 总结与扩展规划

10.1 项目总结

本项目基于 Flutter + Win32 C++ 实现了校车监控管理桌面客户端, 架构分层清晰、模块解耦、符合 Windows 桌面程序开发规范。

原生层稳定完成窗口管理、系统交互、引擎托管；

Flutter 层专注业务界面与逻辑，跨平台能力强；

整体具备高 DPI、明暗主题、插件扩展、消息互通能力，满足校车系统日常使用需求。

10.2 后续扩展规划

1. 硬件对接：新增串口 / 网络模块，原生 C++ 对接校车定位、GPS、门禁硬件；
2. 通道扩展：完善 Platform Channel，实现 Dart 与 C++ 双向高频通信；
3. 多窗口：扩展多子窗口，实现独立弹窗、日志窗口、监控大屏；
4. 权限与安全：增加登录验证、数据加密、日志本地持久化；
5. 打包优化：精简依赖、单文件打包、开机自启配置。